04. Parallel Programming in Vulkan

Objectives of This Lecture

Understanding vulkan’s parallel programming

- Efficient Graphics Command Management with Command Buffers and Pools.
- Enhanced Performance through Multi-Threaded Command Execution.
- Understanding performance advantages and the role of applications in managing thread-related tasks.

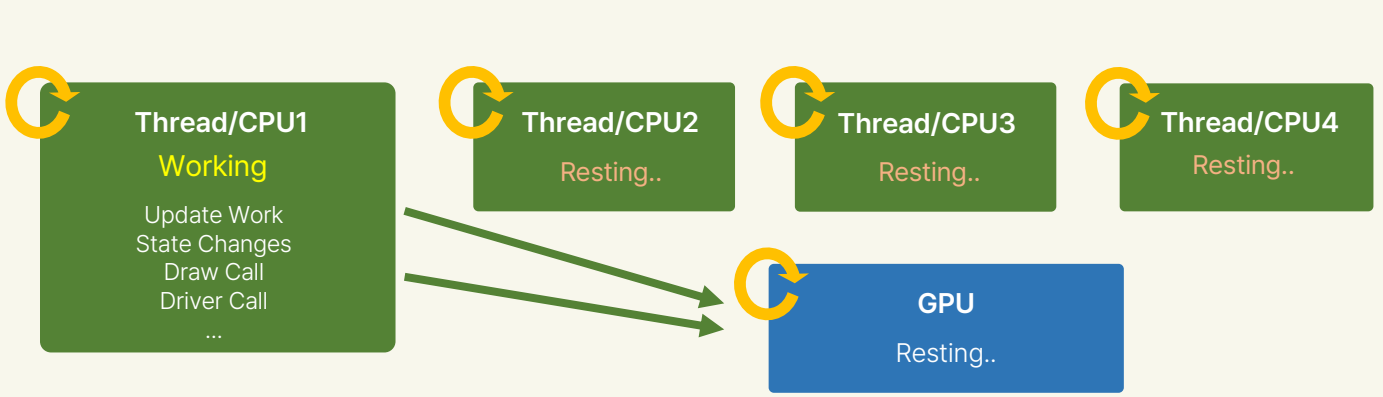
Vulkan’s Parallel Programming Overview

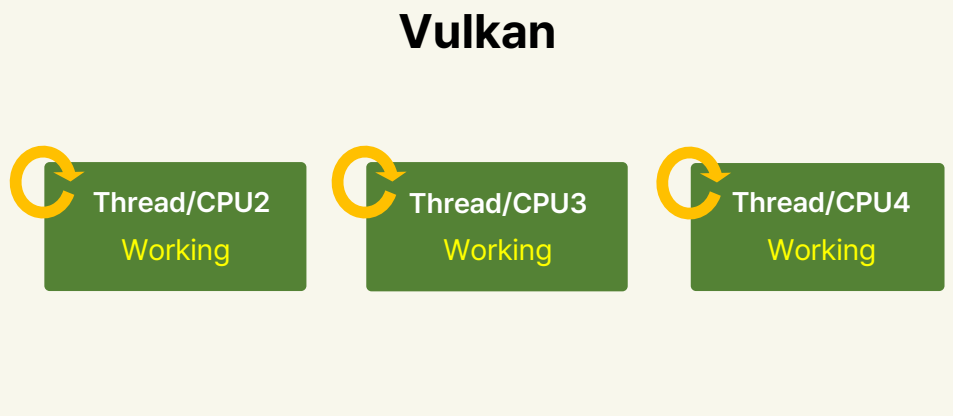
- **Features of Vulkan**
  - The concurrent utilization of GPU cores to process a large volume of data
  - Low-level API designed for multi-core usage
  - Optimizing the parallel processing capabilities of GPUs

- **Multithreading**
  - **OpenGL**
    - Typically, all rendering is done in a single thread (OpenGL is fundamentally single-threaded)
    - While multithreading is possible in OpenGL, it can result in overhead
  - **Vulkan**
    - Allows for the concurrent creation and submission of graphics commands from multiple threads

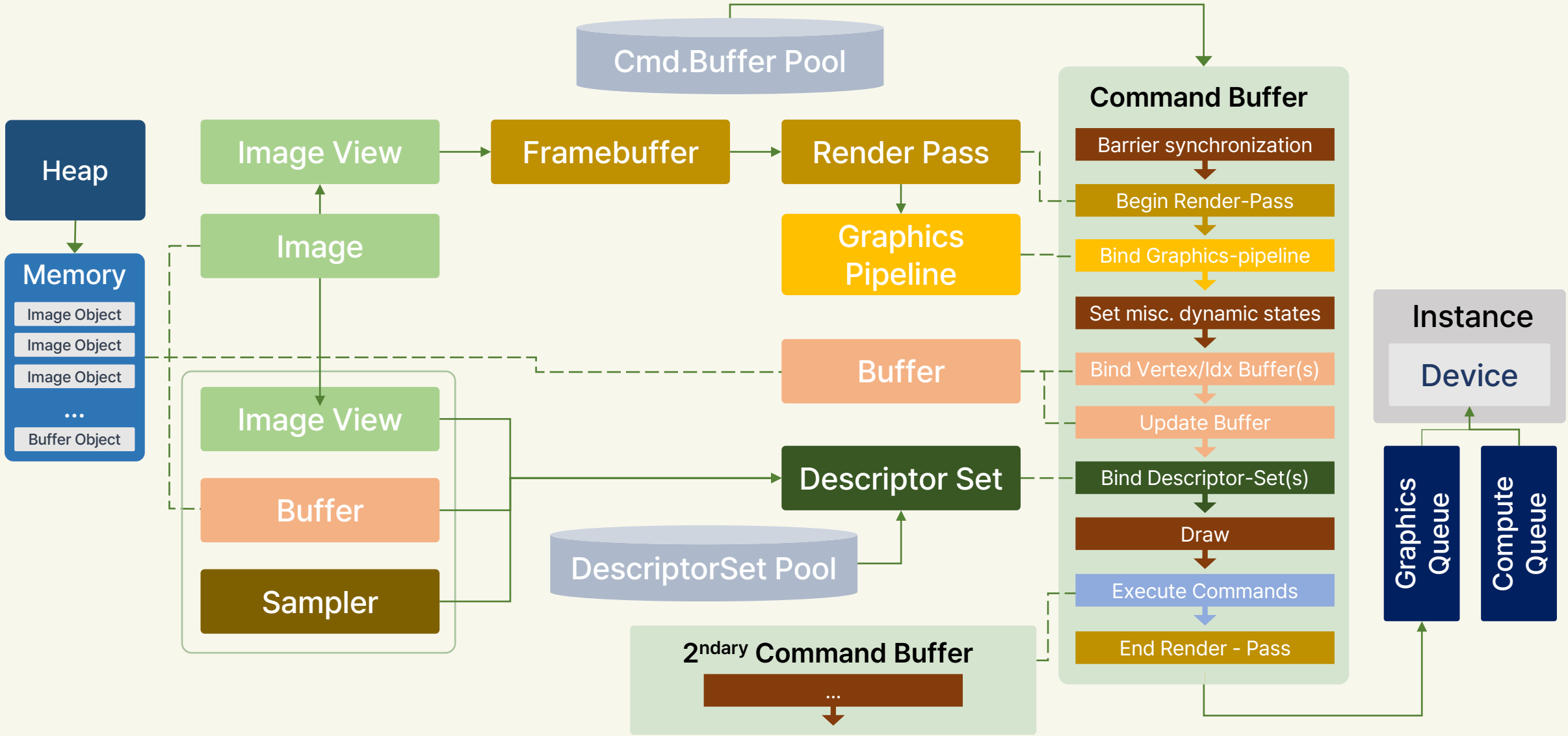
Vulkan's Parallel Programming Overview

- **GPU Enumeration and Resource Allocation**
  - Enumeration of GPUs within a system
  - Allocation of GPU resources
  - Preparation and submission of command buffers to the GPU
  - Memory allocation and read/write operations on the GPU
- **Multithreading works in Vulkan**
  - Vulkan enables independent rendering command sequence creation by multiple threads





Vulkan Components



Command Buffer

- All GPU commands pass through a Command Buffer
- Operations like drawing, memory transfers are recorded in Command Buffer, not executed directly
- Efficiency
  - Commands are submitted together for Vulkan's optimal handling
- Multi-threading
  - Command recording can be done from multiple threads
- Command Buffer allows structured and efficient command management for GPU operations

Command Queue

- Overview of Command Queues in Vulkan as GPU's "execution ports”
- Features
  - Multiple queues for concurrent execution of different command streams
  - Queues accept Command Buffers and execute commands in order
  - Order of execution is defined within a queue but not across different queues
- Importance
  - Allows for simultaneous and organized execution of commands
  - Provides a structured approach to command execution on the GPU

Command Pool

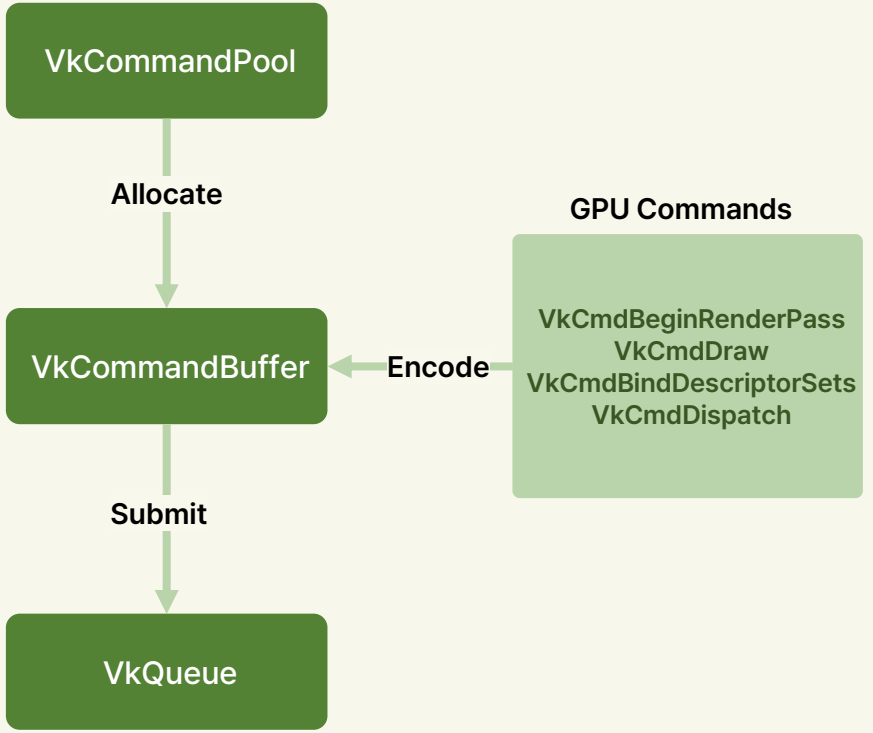
- The Resource Allocator
- Created from `VkDevice` and requires queue family index
- Acts as a background allocator for Command Buffers
- Allocation and Recording
  - Can allocate multiple `VkCommandBuffers` from a pool
  - Only one thread can record commands at a time
- Significance
  - Central to memory allocation for Command Buffers
  - Spreads the cost of resource creation across multiple buffers
- Emphasizes Vulkan's efficient management of GPU resources

Overview of Vulkan Command Execution Phases

01. Allocate a `VkCommandBuffer` from a `VkCommandPool`

02. Record commands into the command buffer, using `VkCmdxxxx` functions

03. submit the command buffer to a `VkQueue`, the execution of the commands begins



Command Pool Creation

Command Pools and Thread Safety in Vulkan

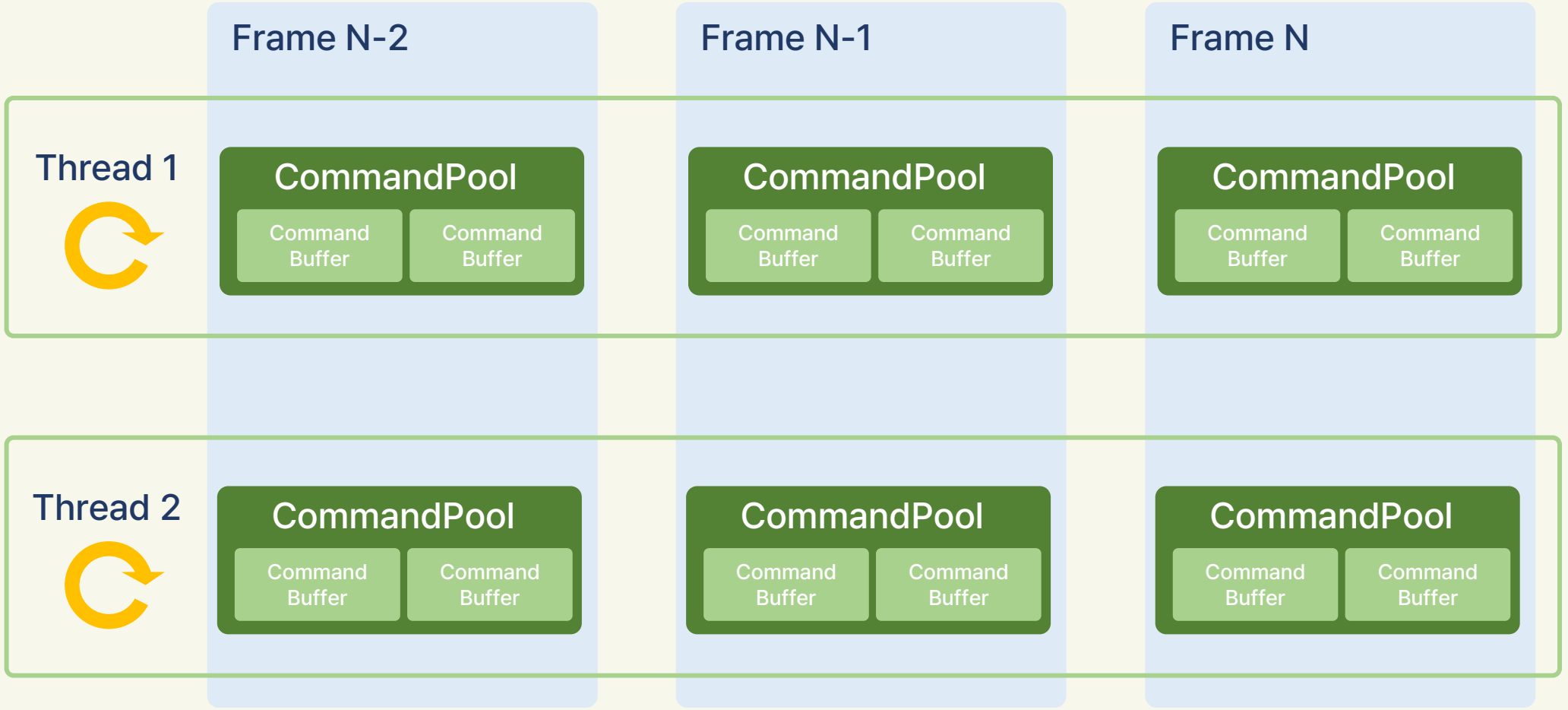
- Essential for recording command buffers across multiple threads
- A single command pool must be externally synchronized
- Key Consideration
  - Non-thread safe: Cannot be accessed simultaneously from multiple threads
- Each thread should have its own Command Pool
  - Alternative: Implement thread synchronization mechanisms
- Importance
  - Ensures efficient and safe handling of command buffers in a multithreaded environment
  - Prevents data races and inconsistencies in command buffer management

Command Pool Creation

Parallel Multithreaded Command Recording

- Utilizing multiple Command Pools for parallel command buffer creation
- Avoids costly locks, enhancing performance in a multithreaded context
- Assign a separate Command Pool to each host-thread
- Enables simultaneous creation of multiple command buffers
- Benefit
  - Lightweight and efficient handling of command submissions across threads
  - Significantly improves performance and scalability in complex applications
- Multiple Command Pools are crucial for effective parallel command recording in Vulkan

Command Pool Creation

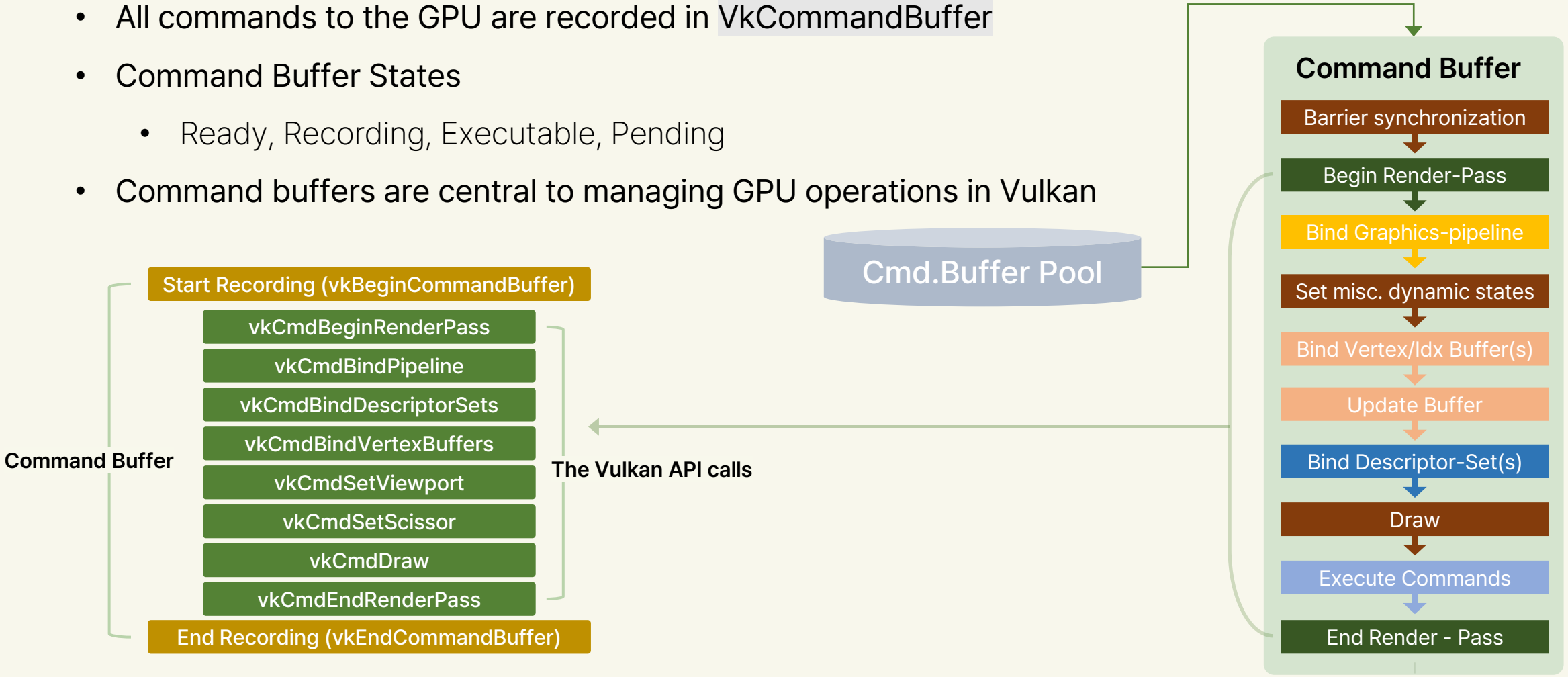


Command Pool Creation

```
for (uint32_t i = 0; i < numThreads; i++) {  // Repeat for 'numThread' times, configuring a 'ThreadData' structure for each thread in each iteration.
ThreadData *thread = &threadData[i];

// Create one command pool for each thread
VkCommandPoolCreateInfo cmdPoolInfo = vks::initializers::commandPoolCreateInfo();
cmdPoolInfo.queueFamilyIndex = swapChain.queueNodeIndex;
cmdPoolInfo.flags = VK_COMMAND_POOL_CREATE_RESET_COMMAND_BUFFER_BIT;
VK_CHECK_RESULT(vkCreateCommandPool(device, &cmdPoolInfo, nullptr, &thread->commandPool)); // For each thread, create a 'VkCommandPool' that manages the command buffers to be used by that thread

// One secondary command buffer per object that is updated by this thread
thread->commandBuffer.resize(numObjectsPerThread);
// Generate secondary command buffers for each thread
VkCommandBufferAllocateInfo secondaryCmdBufAllocateInfo =
vks::initializers::commandBufferAllocateInfo(
thread->commandPool, // Specifies the command pool to allocate the command buffer
VK_COMMAND_BUFFER_LEVEL_SECONDARY, // secondary command buffer creation
thread->commandBuffer.size()); // Number of command buffers to allocate

//Allocate the secondary command buffer by calling the vkAllocateCommandBuffers function.
VK_CHECK_RESULT(vkAllocateCommandBuffers(device, &secondaryCmdBufAllocateInfo, thread->commandBuffer.data()));
```

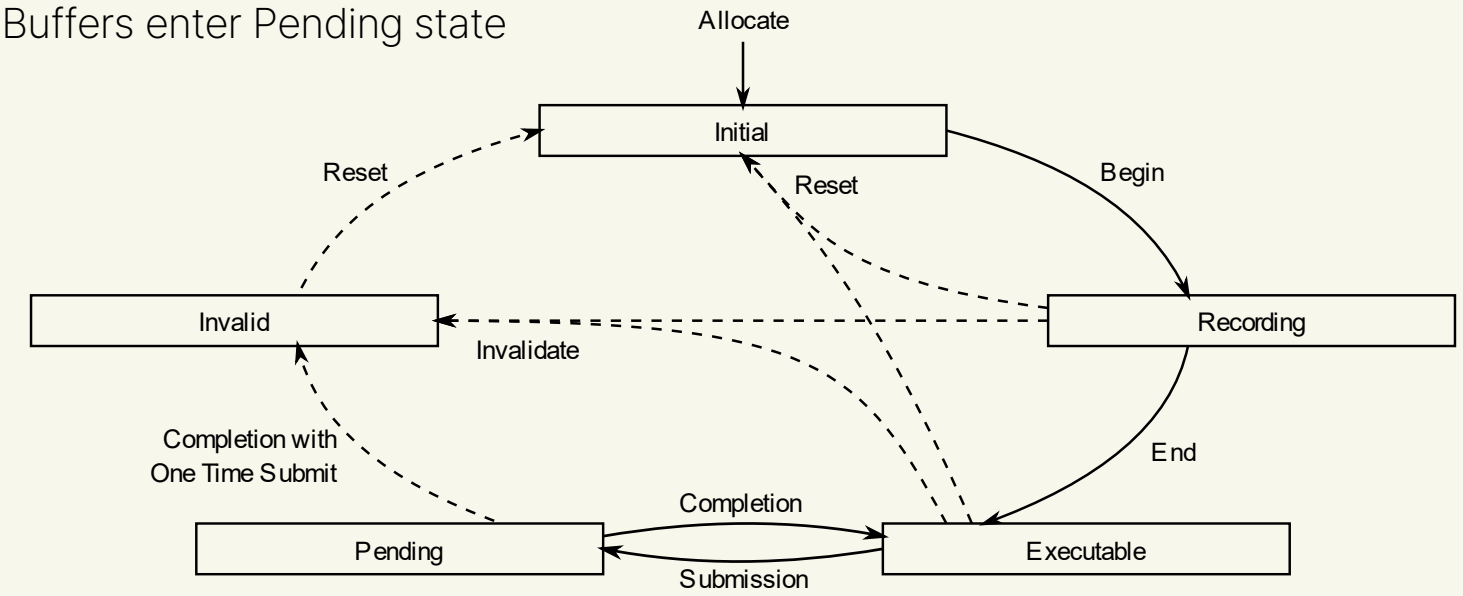
Command Buffer Creation

- All commands to the GPU are recorded in VkCommandBuffer
- Command Buffer States
  - Ready, Recording, Executable, Pending
- Command buffers are central to managing GPU operations in Vulkan



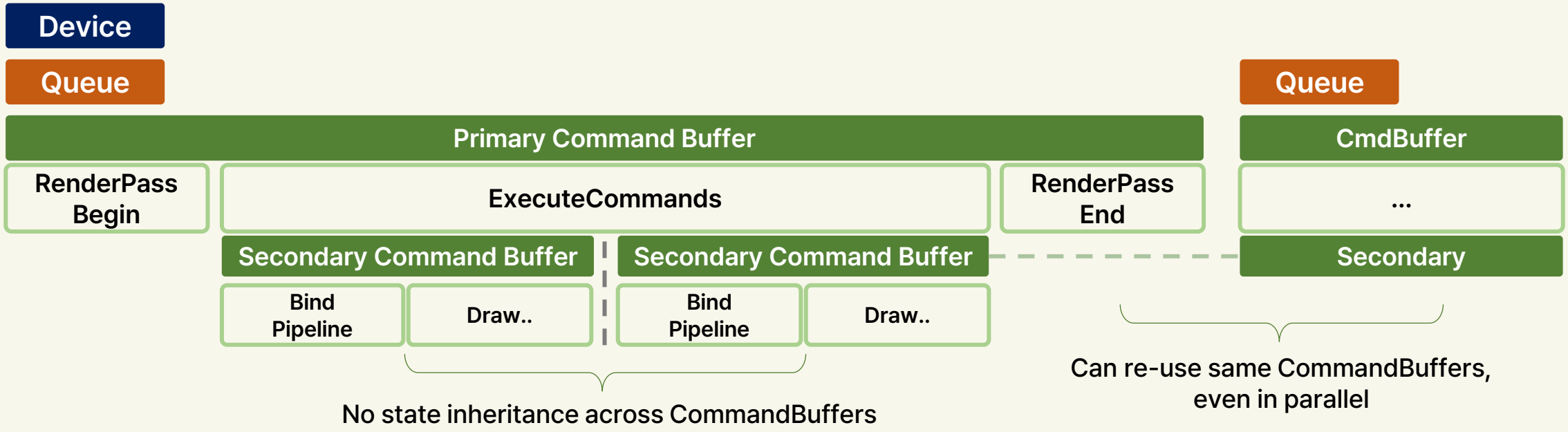
Command Buffer Lifecycle

- No GPU work is executed until the command buffer is submitted to the GPU
- Lifecycle Stages of a Command Buffer
  - Starts in Ready state
  - Transition to Recording state using `vkBeginCommandBuffer()`
  - Input commands using `vkCmdxxx` functions
  - Complete recording with `vkEndCommandBuffer()` transitioning to Executable state
  - Submission to GPU using `vkQueueSubmit()`
    - `vkQueueSubmit` allows submitting multiple command buffers together
  - Post-submission: Command Buffers enter Pending state



Primary and Secondary Command Buffers

- Types of Command Buffers
  - **Primary:** Can be submitted to a queue
  - **Secondary:** Executable within a primary command buffer
- Secondary Command Buffer Usage
  - Inside or outside of Render Passes
  - Not independently submittable, designed for multithreading



Primary and Secondary Command Buffers

- Primary command buffer and Secondary command buffer

```
// primaryCommandBuffer
VkCommandBuffer primaryCommandBuffer;

// Secondary scene command buffers (This code is used to store backdrop and user interface)
struct SecondaryCommandBuffers {
VkCommandBuffer background;
VkCommandBuffer ui;
} secondaryCommandBuffers;
```

```
VkCommandBufferAllocateInfo cmdBufAllocateInfo =     //  Primary Command Buffer
vks::initializers::commandBufferAllocateInfo(
cmdPool,
VK_COMMAND_BUFFER_LEVEL_PRIMARY,
1);
VK_CHECK_RESULT(vkAllocateCommandBuffers(device, &cmdBufAllocateInfo, &primaryCommandBuffer));

// Create additional secondary CBs for background and ui
cmdBufAllocateInfo.level = VK_COMMAND_BUFFER_LEVEL_SECONDARY;      // Secondary Command Buffer
VK_CHECK_RESULT(vkAllocateCommandBuffers(device, &cmdBufAllocateInfo, &secondaryCommandBuffers.background));    // backdrop
VK_CHECK_RESULT(vkAllocateCommandBuffers(device, &cmdBufAllocateInfo, &secondaryCommandBuffers.ui));    // UI
```

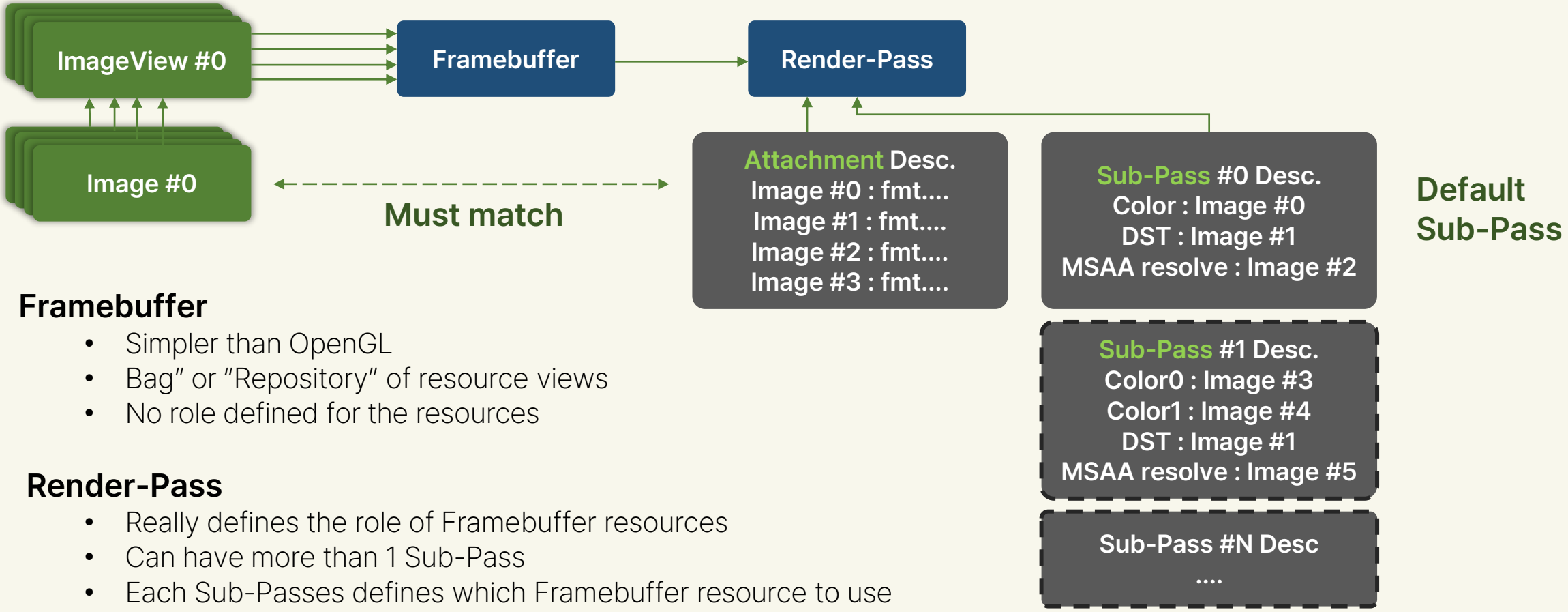
Render Pass

Render Pass in Vulkan

- Fundamental for managing rendering operations
- All rendering occurs within `VkRenderPass`
- Defines the setup or “target” for rendering, specifying the state of the image to be rendered
  - To provide additional information about the state of the rendering image.
- Close relationship between Render Passes and Framebuffers
- Render Pass Execution
  - Setting up images for rendering using Framebuffers
  - Encoding rendering commands between `vkBeginCommandBuffer` and `vkEndCommandBuffer`, with `vkCmdBeginRenderPass` and `vkCmdEndRenderPass` marking the start and end of a render pass

Render Pass

Render Pass in Vulkan



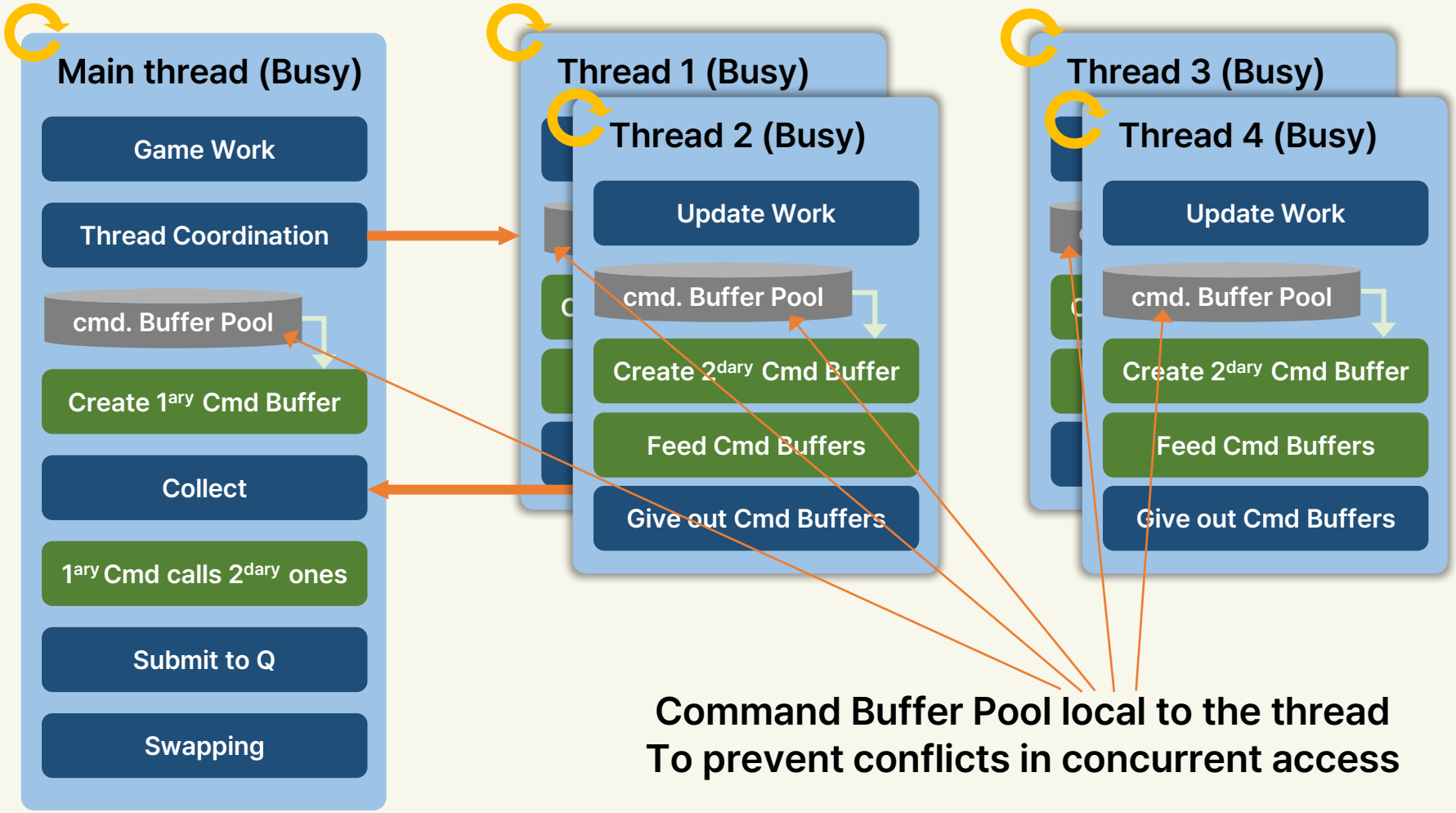
Framebuffer

- Simpler than OpenGL
- Bag" or "Repository" of resource views
- No role defined for the resources

Render-Pass

- Really defines the role of Framebuffer resources
- Can have more than 1 Sub-Pass
- Each Sub-Passes defines which Framebuffer resource to use
- invented for Tilers Arch

Command Buffers and Multi-Threading



Multithreaded Command Buffer Recording

Multithreaded Command Buffer Recording and Resource Management

- Thread Management in Command Buffer Recording
  - Importance of managing memory access and usage across threads for resources related to Command Pool (buffers, descriptor sets)
- Parallel Command Buffer Recording
  - Multiple Command Pools (ideally one per thread) are necessary
  - Enhances efficiency and scalability in complex Vulkan applications

Multithreaded Command Buffer Recording

```
// Builds the secondary command buffer for each thread
void threadRenderCode(uint32_t threadIndex, uint32_t cmdBufferIndex, VkCommandBufferInheritanceInfo inheritanceInfo)  //
{
ThreadData *thread = &threadData[threadIndex];
ObjectData *objectData = &thread->objectData[cmdBufferIndex];

// Check visibility against view frustum using a simple sphere check based on the radius of the mesh
objectData->visible = frustum.checkSphere(objectData->pos, models.ufo.dimensions.radius * 0.5f);

if (!objectData->visible)
{
return;
}

VkCommandBufferBeginInfo commandBufferBeginInfo = vks::initializers::commandBufferBeginInfo();
commandBufferBeginInfo.flags = VK_COMMAND_BUFFER_USAGE_RENDER_PASS_CONTINUE_BIT;
commandBufferBeginInfo.pInheritanceInfo = &inheritanceInfo;

VkCommandBuffer cmdBuffer = thread->commandBuffer[cmdBufferIndex];

VK_CHECK_RESULT(vkBeginCommandBuffer(cmdBuffer, &commandBufferBeginInfo));  //vkBeginCommandBuffer

// ... Record commands (vkCmdxxxx )

VK_CHECK_RESULT(vkEndCommandBuffer(cmdBuffer)); //vkEndCommandBuffer
```

Command Buffer Update - Secondary

```
void updateSecondaryCommandBuffers(VkCommandBufferInheritanceInfo inheritanceInfo)
{
// Secondary command buffer for the sky sphere
VkCommandBufferBeginInfo commandBufferBeginInfo = vks::initializers::commandBufferBeginInfo();
commandBufferBeginInfo.flags = VK_COMMAND_BUFFER_USAGE_RENDER_PASS_CONTINUE_BIT;
commandBufferBeginInfo.pInheritanceInfo = &inheritanceInfo;

VkViewport viewport = vks::initializers::viewport((float)width, (float)height, 0.0f, 1.0f);
VkRect2D scissor = vks::initializers::rect2D(width, height, 0, 0);

/*
Background
*/

VK_CHECK_RESULT(vkBeginCommandBuffer(secondaryCommandBuffers.background, &commandBufferBeginInfo));

vkCmdSetViewport(secondaryCommandBuffers.background, 0, 1, &viewport);
vkCmdSetScissor(secondaryCommandBuffers.background, 0, 1, &scissor);

vkCmdBindPipeline(secondaryCommandBuffers.background, VK_PIPELINE_BIND_POINT_GRAPHICS, pipelines.starsphere);

glm::mat4 mvp = matrices.projection * matrices.view;
mvp[3] = glm::vec4(0.0f, 0.0f, 0.0f, 1.0f);
mvp = glm::scale(mvp, glm::vec3(2.0f));
```

Command Buffer Update - Secondary

```
vkCmdPushConstants(
secondaryCommandBuffers.background,
pipelineLayout,
VK_SHADER_STAGE_VERTEX_BIT,
0,
sizeof(mvp),
&mvp);

models.starSphere.draw(secondaryCommandBuffers.background);

VK_CHECK_RESULT(vkEndCommandBuffer(secondaryCommandBuffers.background));

/*
User interface

With VK_SUBPASS_CONTENTS_SECONDARY_COMMAND_BUFFERS, the primary command buffer's content has to be defined
by secondary command buffers, which also applies to the UI overlay command buffer
*/

VK_CHECK_RESULT(vkBeginCommandBuffer(secondaryCommandBuffers.ui, &commandBufferBeginInfo));

vkCmdSetViewport(secondaryCommandBuffers.ui, 0, 1, &viewport);
vkCmdSetScissor(secondaryCommandBuffers.ui, 0, 1, &scissor);

vkCmdBindPipeline(secondaryCommandBuffers.ui, VK_PIPELINE_BIND_POINT_GRAPHICS, pipelines.starsphere);

drawUI(secondaryCommandBuffers.ui);

VK_CHECK_RESULT(vkEndCommandBuffer(secondaryCommandBuffers.ui));
}
```

Command Buffer Update

```
void updateCommandBuffers(VkFramebuffer frameBuffer)   //  Updates the command buffer. It begins with the primary (secondary command buffers are called within the primary)
{
// Contains the list of secondary command buffers to be submitted
std::vector<VkCommandBuffer> commandBuffers;

VkCommandBufferBeginInfo cmdBufInfo = vks::initializers::commandBufferBeginInfo();

VkClearValue clearValues[2];
clearValues[0].color = defaultClearColor;
clearValues[1].depthStencil = { 1.0f, 0 };

VkRenderPassBeginInfo renderPassBeginInfo = vks::initializers::renderPassBeginInfo();
renderPassBeginInfo.renderPass = renderPass;
// ... RenderPass
renderPassBeginInfo.framebuffer = frameBuffer;

// Set target frame buffer

VK_CHECK_RESULT(vkBeginCommandBuffer(primaryCommandBuffer, &cmdBufInfo));   // Begins the primaryCommandBuffer
```

Command Buffer Update

```
// The primary command buffer does not contain any rendering commands
// These are stored (and retrieved) from the secondary command buffers
vkCmdBeginRenderPass(primaryCommandBuffer, &renderPassBeginInfo, VK_SUBPASS_CONTENTS_SECONDARY_COMMAND_BUFFERS);  // Begins the RenderPass

// Inheritance info for the secondary command buffers
VkCommandBufferInheritanceInfo inheritanceInfo = vks::initializers::commandBufferInheritanceInfo();
inheritanceInfo.renderPass = renderPass;
// Secondary command buffer also use the currently active framebuffer
inheritanceInfo.framebuffer = frameBuffer;

// Update secondary sene command buffers
updateSecondaryCommandBuffers(inheritanceInfo);  // Updates the SecondaryCommandBuffer (updates secondary command buffers related to background and UI)

if (displayStarSphere) {
commandBuffers.push_back(secondaryCommandBuffers.background);
}

// Add a job to the thread's queue for each object to be rendered .  Assigns tasks to each thread and waits until all thread tasks are completed.
for (uint32_t t = 0; t < numThreads; t++)
{
for (uint32_t i = 0; i < numObjectsPerThread; i++)
{
threadPool.threads[t]->addJob([=] { threadRenderCode(t, i, inheritanceInfo); });
}
}
threadPool.wait();
```

Command Buffer Update

```
// Only submit if object is within the current view frustum
// Collects the results from each thread (command buffers for visible objects) and adds them to the command buffer vector
for (uint32_t t = 0; t < numThreads; t++)
{
for (uint32_t i = 0; i < numObjectsPerThread; i++)
{
if (threadData[t].objectData[i].visible)
{
commandBuffers.push_back(threadData[t].commandBuffer[i]);
}
}
}

// Render ui last
if (UIOverlay.visible) {
commandBuffers.push_back(secondaryCommandBuffers.ui);
}

// Execute render commands from the secondary command buffer   (Executes the secondaryCommandBuffer from the primaryCommandBuffer)
vkCmdExecuteCommands(primaryCommandBuffer, commandBuffers.size(), commandBuffers.data());

vkCmdEndRenderPass(primaryCommandBuffer); // Ends the render pass

VK_CHECK_RESULT(vkEndCommandBuffer(primaryCommandBuffer));  //Completes recording of the primaryCommandBuffer
}
```

Queue Submission

Queue Submission Process

- Only one thread can submit commands to a specific queue at a time
- Multiple threads require multiple queues for simultaneous `VkQueueSubmit` operations
- Role of Queues
  - Hardware units in GPU that process Command Buffers
  - Every GPU has multiple queues for executing various command streams
- Separate queues can execute commands simultaneously, enhancing parallel processing capabilities

Queue Types and Functions

- Derived from Queue Families, determining the 'type' and supported command types
- Types include Graphics, Compute, Transfer, Sparse, among others
- Importance of Queue Types
  - Enables specialized processing for different command types
  - Facilitates optimized GPU performance for specific tasks

Queue operations found in VkQueueFlagBits

- VK_QUEUE_GRAPHICS_BIT used for vkCmdDraw* and graphic pipeline commands.
- VK_QUEUE_COMPUTE_BIT used for vkCmdDispatch* and vkCmdTraceRays* and compute pipeline related commands.
- VK_QUEUE_TRANSFER_BIT used for all transfer commands.
- VK_PIPELINE_STAGE_TRANSFER_BIT in the Spec describes “transfer commands”.
- Queue Families with only VK_QUEUE_TRANSFER_BIT are usually for using DMA to asynchronously transfer data between host and device memory on discrete GPUs, so transfers can be done concurrently with independent graphics/compute operations.
- VK_QUEUE_GRAPHICS_BIT and VK_QUEUE_COMPUTE_BIT can always implicitly accept VK_QUEUE_TRANSFER_BIT commands.
- VK_QUEUE_SPARSE_BINDING_BIT used for binding sparse resources to memory with vkQueueBindSparse.
- VK_QUEUE_PROTECTED_BIT used for protected memory.
- VK_QUEUE_VIDEO_DECODE_BIT_KHR and VK_QUEUE_VIDEO_ENCODE_BIT_KHR used with Vulkan Video.

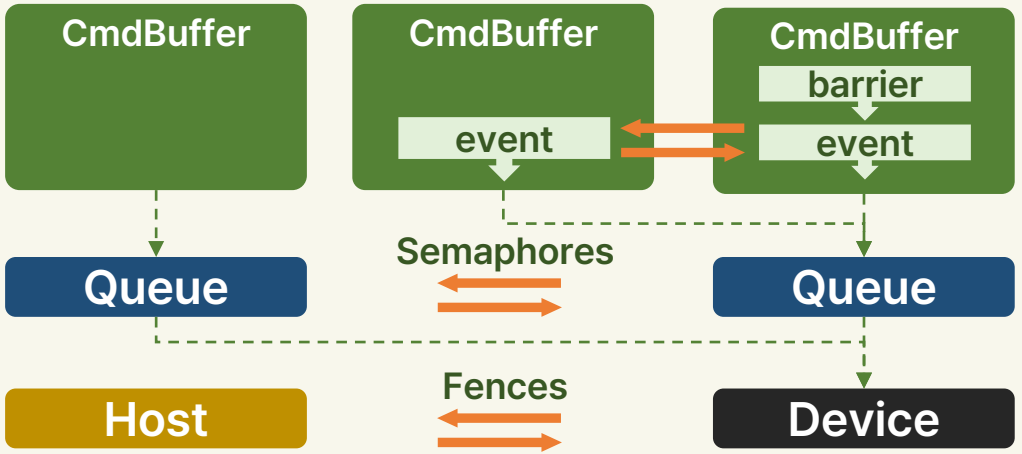
Queue Submission and CmdBuffer State

- Submission Rules
  - Command buffer must be in a Complete state before submission
  - Once a command buffer is submitted, it remains 'alive' and is being consumed by the GPU

- Safety Considerations
  - Unsafe to reset a command buffer while it is still being processed by the GPU
  - Must confirm the completion of all commands in a command buffer before resetting and reusing it
  - Use `vkResetCommandBuffer()` for resetting

Asynchronous Execution and Synchronization

- Asynchronous Execution
  - Upon submission, commands in the command buffer are executed asynchronously by the GPU
- Synchronization Necessity
  - Ensures proper order and completion of tasks between CPU and GPU or among different queues
- Synchronization
  - **Semaphores**: Synchronize tasks between or within queues
  - **Events and Barriers**: Synchronize within a command buffer or among buffers in the same queue.
  - **Fences**: Synchronization between device (GPU) and host (CPU).



Queue Submission

```
void draw()  // Implements Vulkan's rendering process, waits for all command buffers to be ready, then submits them to the queue for rendering
{
// Wait for fence to signal that all command buffers are ready
VkResult fenceRes;
do {
fenceRes = vkWaitForFences(device, 1, &renderFence, VK_TRUE, 100000000);  // Calls vkWaitForFences to wait for all command buffers from the previous frame to be processed by the GPU (synchronization between the CPU and GPU)
} while (fenceRes == VK_TIMEOUT);
VK_CHECK_RESULT(fenceRes);
vkResetFences(device, 1, &renderFence);  //Calls vkResetFences to reset the used fence.

VulkanExampleBase::prepareFrame();   // Prepares for the next frame

updateCommandBuffers(frameBuffers[currentBuffer]);  // Updates the command buffers for the current frame, recording new commands depending on what is being rendered

submitInfo.commandBufferCount = 1;
submitInfo.pCommandBuffers = &primaryCommandBuffer;

//  Uses vkQueueSubmit to submit the prepared command buffer primaryCommandBuffer to the queue, signaling the GPU to start rendering
// The submitInfo structure used for submission contains information related to the command buffer being submitted
// Uses 'renderFence' along with the submission to track the completion of GPU work
VK_CHECK_RESULT(vkQueueSubmit(queue, 1, &submitInfo, renderFence));

VulkanExampleBase::submitFrame();
}
```

To be continued

Ray Tracing with Vulkan

- Types of Shaders
- Adding Shader
- Shader Binding Table